HypG v2.0.1: A Complete Safe-Rust Post-Quantum Cryptographic Suite

Extended Technical Report on Design, Formal Specification, and Evaluation

HypG Project

Independent research prototype

Abstract. We present HypG v2.0.1, a from-scratch, zero-external-dependency post-quantum cryptographic suite written in Rust with `#![forbid(unsafe_code)]` in every library. The suite contains a Module-LWE key-encapsulation mechanism at module rank four over $\mathbb{Z}_{4096}[X]/(X^{256}+1)$; a portable BLAKE3 hash, XOF, keyed-hash, and derive-key layer; a ChaCha20 plus keyed-BLAKE3 Encrypt-then-MAC authenticated-encryption scheme with byte-identical one-shot and streaming interfaces; and a stateless hash-based signature scheme with FORS 35×9, WOTS+ $w = 16$, and a 17-layer XMSS hypertree. The signature wire format carries the separate **2.0.1** revision: explicit one-byte PRF/F domain tags and FIPS-205-style promotion of odd WOTS+ L-tree nodes.

The paper is deliberately both a design document and a normative technical report. It gives the exact algorithms and wire layouts, formal definitions and correctness lemmas, the complete domain registry, the public API contracts, the frozen known-answer-test inventory, and a controlled evaluation. The recorded release has 146 deterministic tests passing in debug, release, and on the Rust 1.70 MSRV; four frozen KAT files totalling 234,114 bytes; a 14-suite adversarial harness; a 20,000-trial failure sweep; a full 11,520-bit sweep of one test ciphertext; Miri coverage of the KEM, AEAD, and DS bounds tests; and byte-identical KAT output on x86-64 and `qemu-aarch64`. The White Paper companion gives the design rationale and threats-to-validity discussion; the Yellow Paper gives the same normative material in a standalone specification format.

None of this is a security proof or an audit. The KEM's exact FO-style transform is not covered by a published theorem; the lattice hardness of the chosen parameters has not been confirmed by an external estimator; the signature composition has no exact signing-volume bound; the AEAD composition has no reduction for its exact instantiation; and there is no external audit, formal verification, `dudect`-grade timing study, or power/EM/fault campaign. HypG is a research prototype and must not be deployed to protect real assets.

Keywords: post-quantum cryptography · Module-LWE · Fujisaki–Okamoto · hash-based signatures · FORS · WOTS+ · XMSS · BLAKE3 · ChaCha20 · authenticated encryption · safe Rust · reproducible evaluation.

Table of Contents

1 Introduction . . . 3
2 Background and related work . . . 4
3 Design principles and architecture . . . 5
4 Notation and mathematical preliminaries . . . 7
5 The CA-PRF expander . . . 7
6 The Module-LWE KEM . . . 8
7 Hash, XOF, and KDF layer . . . 12
8 Authenticated encryption . . . 13
9 Hash-based signatures . . . 14
10 Domain registry . . . 19
11 Implementation assurance . . . 20
12 Threat model and failure semantics . . . 21
13 Evaluation . . . 22
14 Security analysis and open problems . . . 25
15 Threats to validity . . . 26
16 Related work . . . 27
17 Road to a defensible profile . . . 27
18 Conclusion . . . 27
A Parameter register . . . 28
B Domain and API summary . . . 28
C Artifact and reproducibility notes . . . 29
D Benchmark detail . . . 29
E Comparison with FIPS 203 and FIPS 205 . . . 30
F Symbol glossary . . . 30

1 Introduction

Public-key cryptography deployed today is vulnerable to a cryptographically relevant quantum computer: Shor's algorithm solves integer factorisation and discrete logarithms in polynomial time. Symmetric primitives are less affected, but their effective security halves under Grover search. A post-quantum suite must therefore combine quantum-resistant public-key constructions with conservative symmetric cryptography and a disciplined implementation.

HypG is an experiment in building the *whole* suite rather than a single primitive: key establishment, hashing, authenticated encryption, and signatures, sharing one small implementation core. The project imposes unusually strict engineering constraints: no external dependencies, no `unsafe` code, `no_std` cryptographic libraries, fixed public loop bounds, explicit little-endian encodings, deterministic test vectors, and mechanically checked CI gates.

The current release is **v2.0.0** with a **DS v2.0.1** wire revision. It is a research prototype. The report deliberately separates (i) what the code does, (ii) what the test suite establishes, and (iii) what would still be required for a security claim. This extended report is a companion to the White and Yellow Papers; it is not a peer-reviewed publication, an audit, or a certification.

1.1 Contributions

The contributions are:

1. **A complete, dependency-free implementation.** A `no_std`, no-allocation cryptographic core and four protocol pillars, with all dependency edges confined to workspace path dependencies and all library crates under `#![forbid(unsafe_code)]`.
2. **A precise v2.0 parameter profile.** A rank-four Module-LWE KEM at lattice dimension 1024 with a power-of-two modulus $q = 4096$, and an SLH-DSA-256f-shaped signature profile with $n = 32$, $h = 68$, $d = 17$, $h' = 4$, FORS $(35, 9)$, and WOTS+ $w = 16$.
3. **A documented DS 2.0.1 revision.** Explicit one-byte PRF/F tags remove a structural collision between the two 64-byte keyed-hash inputs, and odd WOTS+ L-tree nodes are promoted unchanged in the FIPS 205 style.
4. **An honest treatment of the CA-PRF.** The cellular automaton is presented as a branchless mixing and domain-separation stage whose uniformity argument rests on the final BLAKE3 XOF, not as an independently proven hardness assumption.
5. **A corrected KEM correctness analysis.** The compression residuals from c_1 and c_2 are included explicitly, yielding a working standard deviation of approximately 70 and a margin of approximately 14.6σ against the 1024 decision boundary. This is a heuristic estimate, not a formal failure bound.
6. **A byte-identical streaming AEAD.** The streaming front uses the same subkeys, MAC input, counter schedule, and tag as the one-shot construction; only the scheduling differs.

7. **A reproducible evaluation record.** Frozen KAT checksums, cross-architecture equality, a deterministic red-team harness, a local lattice lab, Miri subsets, and a controlled release benchmark are recorded with commands and machine details.
8. **A complete normative specification.** The report includes the exact algorithms, byte layouts, domain registry, API contracts, and KAT inventory, not merely a high-level design narrative.
9. **Explicit non-claims and an audit agenda.** The report names the exact proof, estimator, side-channel, and operational gaps that stand between the current artifact and a deployable system.

1.2 What this report does not claim

This report does *not* claim that HypG is IND-CCA2 secure, UF-CMA/SUF-CMA secure, or a secure AEAD in the sense of a published reduction for its exact composition. It does not claim NIST Category-5 security, FIPS compliance, certification, or production readiness. It does not claim constant-time execution at the machine-code level, and it does not claim resistance to power, electromagnetic, or fault attacks. KAT vectors pin behaviour; they are not security evidence.

1.3 Organisation

Section 2 reviews the relevant background. Section 3 states the design constraints. Section 3 describes the architecture and design constraints. Section 4 fixes notation. Sections 6–9 describe the four pillars. Section 11 discusses implementation assurance. Section 13 records the controlled evaluation. Section 14 analyses assumptions and open problems. Section 15 discusses threats to validity. Section 16 relates HypG to prior work. Section 17 gives the path to a defensible profile.

2 Background and related work

2.1 Post-quantum cryptography

The NIST post-quantum standardisation process produced ML-KEM (FIPS 203) [1], ML-DSA (FIPS 204) [2], and SLH-DSA (FIPS 205) [3]. ML-KEM is a Module-LWE KEM; ML-DSA is a Module-LWE/Module-SIS signature scheme; SLH-DSA is a stateless hash-based signature scheme derived from SPHINCS+ [8]. HypG does not implement any of these standards verbatim. It uses the same broad mathematical families and some of the same dimension choices, but with different primitives, encodings, domain separation, and wire formats.

2.2 Module-LWE

Module-LWE generalises ring-LWE by working over R_q^k. For a public matrix A and secret/error vectors s, e drawn from a small distribution, the public key is $b = As + e$. Encryption uses a fresh small vector r and errors e_1, e_2:

$$c_1 = A^T r + e_1, \qquad c_2 = \langle b, r \rangle + e_2 + \mathsf{Encode}(m).$$

The decryption noise is $\langle e, r \rangle + e_2 - \langle s, e_1 \rangle$; correctness requires it to remain inside the decoding region. The concrete hardness depends on the ring degree, module rank, modulus, noise distribution, and number of samples. The community lattice-estimator [10] is the standard tool for estimating the cost of known attacks; it has not been run on HypG's exact parameters.

2.3 Hash-based signatures

SLH-DSA and SPHINCS+ combine a few-time FORS signature with a hypertree of XMSS/WOTS+ one-time signatures. The hash function is tweaked by a public seed and an address so that every hash call is domain-separated. HypG follows this architecture with BLAKE3 as the tweakable hash, a custom 32-byte address, and a deterministic randomizer. Determinism is useful for reproducible vectors but removes fresh-randomness hedging; the security consequences are discussed in Section 9.

2.4 Symmetric primitives

BLAKE3 [4] is a tree-structured hash and XOF with a 32-byte keyed mode and a reference `derive_key` mode. ChaCha20 [5] is a 20-round ARX stream cipher with a 96-bit nonce and a 32-bit block counter. HypG uses both as designed: BLAKE3 for hashing, expansion, key derivation, and MACs; ChaCha20 for confidentiality. The AEAD is an Encrypt-then-MAC composition rather than a standardised AEAD algorithm.

3 Design principles and architecture

3.1 Five constraints

Five constraints shape every component.

1. **No external dependencies.** Every dependency edge is a workspace path dependency. The supply-chain surface is therefore the Rust toolchain and the repository itself, not a transitive crate graph. `cargo tree` is part of the release gate.
2. **No `unsafe`.** Every library crate uses `#![forbid(unsafe_code)]`. Arithmetic and conversion paths are written to be total where possible. The cost is scalar code and best-effort, rather than volatile, zeroization.

Component	Code version	Wire/spec revision	Frozen vectors
Workspace	2.0.0	—	—
`hypg-kem`	2.0.0	KEM v2.0	`kats/v2.0.0.txt`
`hypg-symm`	2.0.0	Symm v2.0	`kats-symm/v2.0.0.txt`
`hypg-hash`	2.0.0	Hash v2.0	`kats-hash/v2.0.0.txt`
`hypg-ds`	2.0.0	**DS v2.0.1**	`kats-ds/v2.0.1.txt`

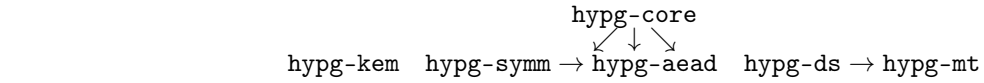
Table 1. Version identity. The literal `HYP-G/v07/...` strings are frozen domain separators, not a statement that the parameter set is v0.7.

3. `no_std` **cryptographic libraries.** The core, KEM, Symm, AEAD, DS, and hash crates do not require an allocator, a filesystem, or an operating system. Threads, files, and allocation live only in `std` crates.
4. **Fixed public geometry.** Protocol loops have public bounds. Secret-dependent branches, secret-dependent array indices, and secret-dependent early returns are avoided in the cryptographic paths.
5. **Reproducible evidence.** Deterministic KATs, a frozen checksum inventory, an adversarial harness, a lattice lab, cross-architecture equality, and a recorded benchmark are part of the artifact.

These are engineering constraints, not security proofs. In particular, source-level branchlessness does not imply machine-code constant-time execution, and safe Rust does not imply resistance to microarchitectural leakage.

3.2 Component graph

The dependency graph is acyclic:



`hypg-core` supplies portable BLAKE3, ChaCha20, the CA-PRF expander, polynomial arithmetic over $\mathcal{R}_q$, and constant-time helpers. `hypg-hash` is a thin API over the core hash functions. `hypg-kat`, `hypg-bench`, `hypg-redteam`, `hypg-lattice`, and `hypg-cli` are `std` harnesses or operator tools.

3.3 Version identity

3.4 Boundary decisions

Threading is deliberately excluded from the `no_std` crates. `hypg-mt` uses `std::thread::scope` and owns all worker results; deterministic assembly makes its output byte-identical to the serial signer. SIMD intrinsics are excluded by the `unsafe` policy and portable SIMD is not stable on the MSRV; the implementation is scalar and relies on the compiler only for ordinary code generation.

4 Notation and mathematical preliminaries

4.1 Bytes and little-endian encoding

A byte is an element of $\{0, \ldots, 255\}$. For an integer x with $0 \le x < 2^t$, $\mathrm{LE}_t(x)$ is the $t/8$-byte encoding with the least-significant byte first. All multi-byte protocol integers are little-endian unless stated otherwise. Byte ranges are half-open: $x[a : b]$ denotes bytes a through $b - 1$. The symbol $\|$ denotes concatenation. No implicit padding is inserted.

4.2 Ring and distributions

Let

$$\mathcal{R}_q = \mathbb{Z}_q[X]/(X^{256} + 1), \qquad q = 4096 = 2^{12}.$$

An element $a \in \mathcal{R}_q$ is represented by $(a_0, \ldots, a_{255})$ with $a_i \in [0, q)$. Addition and subtraction are coefficient-wise modulo q; multiplication is negacyclic:

$$(a * b)_k = \sum_{\substack{i+j=k \\ 0 \le i,j < 256}} a_i b_j - \sum_{\substack{i+j=k+256 \\ 0 \le i,j < 256}} a_i b_j \pmod q.$$

The distribution CBD(2) uses four independent uniform bits per coefficient:

$$v = (b_0 + b_1) - (b_2 + b_3), \qquad b_i \in \{0, 1\}.$$

The support is $\{-2, -1, 0, 1, 2\}$ with probabilities $1/16, 4/16, 6/16, 4/16, 1/16$; the mean is zero and the variance is one.

4.3 Hash notation

Let $\mathsf{H}(x)$ be unkeyed BLAKE3-256, $\mathsf{H}_k(x)$ keyed BLAKE3-256 with 32-byte key k, and $\mathsf{XOF}(x, L)$ the first L bytes of the BLAKE3 XOF. Define

$$\begin{aligned} \mathrm{expand}(x, d, L) &= \mathsf{XOF}(x \,\|\, d, L), \\ \mathrm{kdf}(d, \mathit{parts}, L) &= \mathsf{XOF}(d \,\|\, \mathit{parts}_0 \,\|\, \mathit{parts}_1 \,\|\, \cdots, L), \\ \mathrm{derive}(c, k) &= \mathrm{BLAKE3.derive_key}(c, k). \end{aligned}$$

Neither `expand` nor `kdf` inserts length separators; all protocol callers use fixed-length parts or a single variable-length part with a distinct domain.

5 The CA-PRF expander

5.1 State and round

Definition 1 (CA state and rule). *A CA state is $S \in (\mathbb{Z}_{2^{64}})^{32}$, serialized as 32 little-endian 64-bit words (256 bytes). A rule is $R \in (\mathbb{Z}_{2^{64}})^8$, serialized as eight little-endian words (64 bytes). The public constants are*

$$\begin{aligned} \mathrm{RC_BASE} &= 0x9E3779B97F4A7C15, \\ \mathrm{RC_STRIDE} &= 0xBF58476D1CE4E5B9, \\ \mathrm{DIFF_MUL} &= 0xD6E8FEB86659FD93. \end{aligned}$$

Definition 2 (CA round). *For round $t \in [0, 32)$ and word $i \in [0, 32)$, with all arithmetic modulo 2^{64}:*

$$
\begin{aligned}
rc &= \mathrm{RC_BASE} + (t+1) \cdot \mathrm{RC_STRIDE},\\
a &= S_i, \quad b = S_{(i+1) \bmod 32}, \quad c = S_{(i+5) \bmod 32}, \quad d = S_{(i+17) \bmod 32},\\
rk &= R_{i \bmod 8} \oplus rc \oplus (i \cdot \mathrm{RC_STRIDE}),\\
t_1 &= a \oplus \mathrm{ROTL}(b, 13) \oplus \mathrm{ROTL}(c, 7),\\
t_2 &= (t_1 \wedge d) \oplus ((\neg t_1) \wedge b),\\
t_3 &= t_2 + (rk \oplus (i + t)),\\
t_4 &= t_3 \oplus \mathrm{ROTL}(t_3, 29),\\
t_5 &= (t_4 \cdot \mathrm{DIFF_MUL}) \oplus \mathrm{ROTL}(d, 11),\\
S'_i &= t_5 \oplus \mathrm{ROTL}(S_{(i+16) \bmod 32}, 17) \oplus rc.
\end{aligned}
$$

The round is applied 32 times.

5.2 CA-PRF definition

Definition 3 (CA-PRF). *For a 32-byte seed x, public domain d, and output length L:*

$$
\begin{aligned}
init &= \mathsf{XOF}(x \,\|\, d \,\|\, \texttt{"-init"}, 256),\\
rule &= \mathsf{XOF}(x \,\|\, d, 64),\\
S_0 &= \mathrm{LE}_{64}^{-1}(init),\\
S_{32} &= \mathrm{Round}^{32}(S_0; rule),\\
out &= \mathsf{XOF}(d \,\|\, \mathrm{LE}_{64}(S_{32}) \,\|\, d \,\|\, \texttt{"-out"}, L).
\end{aligned}
$$

Remark 1 (Security role). The final BLAKE3 XOF is the security-relevant expander. If BLAKE3 is a secure XOF or random oracle, the output is pseudorandom regardless of the CA. The CA is a deterministic mixing and domain-separation stage, not an independently proven hardness assumption.

Lemma 1 (Prefix consistency). *For fixed (x, d) and $L_1 \le L_2$, the first L_1 bytes of* CA-PRF(x, d, L_2) *equal* CA-PRF(x, d, L_1).

Proof. The state and all preceding stages are independent of L; the final XOF is prefix-consistent.

6 The Module-LWE KEM

6.1 Parameters and sizes

The power-of-two modulus differs from ML-KEM-1024's 3329; it enables branchless mask-only reduction and changes the lattice profile. ML-KEM hardness estimates cannot be transferred without a fresh estimator run.

Symbol	Value	Meaning
n	256	ring degree
k	4	module rank; dimension 1024
q	$2^{12} = 4096$	modulus
η	2	CBD(2) for s, e, r, e_1, e_2
m	32 bytes	FO message; one bit per coefficient
pk	1568 bytes	$seed_A(32) \,\|\|\, \mathsf{Pack}_{12}(b)(1536)$
ct	1440 bytes	$\mathsf{Pack}_{10}(c_1)(1280) \,\|\|\, \mathsf{Pack}_5(c_2)(160)$
sk	1632 bytes	$seed(32) \,\|\|\, fake(32) \,\|\|\, pk$
ss	32 bytes	XOF output

Table 2. KEM parameters and sizes.

6.2 Encoding and compression

Definition 4 (Encode/Decode). *For a 32-byte message m:*

$$\mathsf{Encode}(m)_i = 2048 \cdot ((m_{\lfloor i/8 \rfloor} \gg (i \bmod 8)) \wedge 1).$$

For $v \in \mathcal{R}_q$:

$$\mathsf{Decode}(v)_i = ((((v_i + 1024) \bmod q) \gg 11) \wedge 1)\,,$$

with output bits packed least-significant first.

Definition 5 (Compression). *For $c \in [0, q)$:*

$$\begin{aligned}
\mathrm{Compress}_{10}(c) &= ((c + 2) \gg 2) \wedge 0x3FF, \\
\mathrm{Decompress}_{10}(c) &= (c \wedge 0x3FF) \ll 2, \\
\mathrm{Compress}_5(c) &= ((c + 64) \gg 7) \wedge 0x1F, \\
\mathrm{Decompress}_5(c) &= (c \wedge 0x1F) \ll 7.
\end{aligned}$$

Lemma 2 (Residual ranges). *For $c \in [0, q)$, let*

$$\begin{aligned}
\Delta_1 &= \mathrm{Decompress}_{10}(\mathrm{Compress}_{10}(c)) - c \pmod q, \\
\Delta_2 &= \mathrm{Decompress}_5(\mathrm{Compress}_5(c)) - c \pmod q.
\end{aligned}$$

In the signed range, $\Delta_1 \in \{-1, 0, 1, 2\}$ and $\Delta_2 \in \{-63, \ldots, 64\}$.

Proof. Write $c = 4a + r$ with $0 \le r < 4$; the 10-bit quotient is a for $r \le 1$ and $a + 1$ for $r \ge 2$, giving $-r$ or $4 - r$. For the 5-bit case, write $c = 128a + r$; the quotient is a for $r < 64$ and $a + 1$ for $r \ge 64$, giving $-r$ or $128 - r$.

6.3 Key generation

Algorithm 1: $\mathsf{KeyGen}(seed)$

1: Input: a 32-byte seed with at least 256 bits of entropy.
2: $seed_A \leftarrow$ CA-PRF($seed$, "HYP-G/v07/seed-a", 32).
3: $A \leftarrow$ ParseMatrix(CA-PRF($seed_A$, "HYP-G/v07/poly-a", 6144)).
4: $s \leftarrow$ ParseCBDVector(CA-PRF($seed$, "HYP-G/v07/sec-s", 512)).
5: $e \leftarrow$ ParseCBDVector(CA-PRF($seed$, "HYP-G/v07/sec-e", 512)).
6: For $i = 0, \ldots, 3$: $b_i \leftarrow \sum_{j=0}^{3} A_{i,j} * s_j + e_i$ in $\mathcal{R}_q$.
7: $pk \leftarrow seed_A \,\|\, \mathsf{Pack}_{12}(b_0) \,\|\, \cdots \,\|\, \mathsf{Pack}_{12}(b_3)$.
8: $fake \leftarrow$ CA-PRF($seed$, "HYP-G/v07/fake", 32).
9: $sk \leftarrow seed \,\|\, fake \,\|\, pk$.
10: Return (pk, sk); best-effort erase intermediate secrets.

6.4 Encapsulation

Algorithm 2: $\mathsf{Encaps}(pk, m)$

1: Input: a 1568-byte public key and a 32-byte FO message m. The caller MUST supply a fresh CSPRNG output for m on every call.
2: Parse pk as $(seed_A, b_0, \ldots, b_3)$.
3: $A \leftarrow$ ParseMatrix(CA-PRF($seed_A$, "HYP-G/v07/poly-a", 6144)).
4: $rseed \leftarrow \mathsf{H}(m \,\|\, pk \,\|$ "HYP-G/v07/r-seed").
5: $r \leftarrow$ ParseCBDVector(CA-PRF($rseed$, "HYP-G/v07/rand-r", 512)).
6: $e_1 \leftarrow$ ParseCBDVector(CA-PRF($rseed$, "HYP-G/v07/rand-e1", 512)).
7: $e_2 \leftarrow$ CBD(CA-PRF($rseed$, "HYP-G/v07/rand-e2", 128)).
8: For $i = 0, \ldots, 3$: $c_{1,i} \leftarrow \sum_{j=0}^{3} A_{j,i} * r_j + e_{1,i}$.
9: $c_2 \leftarrow \sum_{i=0}^{3} b_i * r_i + e_2 + \mathsf{Encode}(m)$.
10:

$$ct \leftarrow \mathsf{Pack}_{10}(\mathrm{Compress}_{10}(c_{1,0})) \,\|\, \cdots \,\|\, \mathsf{Pack}_{10}(\mathrm{Compress}_{10}(c_{1,3})) \,\|\, \mathsf{Pack}_{5}(\mathrm{Compress}_{5}(c_2)).$$

11: $ss \leftarrow \mathsf{XOF}($"HYP-G/v07/ss" $\,\|\, m \,\|\, ct \,\|\, \mathsf{H}(pk), 32)$.
12: Return (ct, ss).

6.5 Decapsulation

Algorithm 3: $\mathsf{Decaps}(sk, ct)$

1: Parse sk as $(seed, fake, pk)$.
2: Parse pk and expand A as in encapsulation.
3: $s \leftarrow$ ParseCBDVector(CA-PRF($seed$, "HYP-G/v07/sec-s", 512)).
4: Unpack and decompress ct into (c_1, c_2).
5: $v \leftarrow c_2 - \sum_{i=0}^{3} s_i * c_{1,i}$.
6: $m' \leftarrow \mathsf{Decode}(v)$.
7: Recompute ct' exactly as in Algorithm 2 from (pk, m').
8: $ok \leftarrow$ ct_eq(ct', ct).
9: $ss_{\mathrm{ok}} \leftarrow \mathsf{XOF}($"HYP-G/v07/ss" $\,\|\, m' \,\|\, ct \,\|\, \mathsf{H}(pk), 32)$.
10: $ss_{\mathrm{rej}} \leftarrow \mathsf{XOF}($"HYP-G/v07/reject" $\,\|\, fake \,\|\, ct, 32)$.
11: $ss \leftarrow$ ct_select($ok, ss_{\mathrm{ok}}, ss_{\mathrm{rej}}$).
12: Return ss; the function never returns an error.

Offset	Length	Field in *pk*
0	32	$seed_A$
32	384	packed b_0
416	384	packed b_1
800	384	packed b_2
1184	384	packed b_3

Offset	Length	Field in *ct*
0	320	packed compressed $c_{1,0}$
320	320	packed compressed $c_{1,1}$
640	320	packed compressed $c_{1,2}$
960	320	packed compressed $c_{1,3}$
1280	160	packed compressed c_2

Table 3. KEM public-key and ciphertext layouts. The secret key is $seed(32) \,\|\, fake(32) \,\|\, pk(1568)$.

6.6 Wire layout

6.7 Correctness

Lemma 3 (Noise identity). *For a ciphertext produced by Algorithm 2 and unpacked by Algorithm 3, before compression effects,*

$$c_2 - \langle s, c_1 \rangle = \mathsf{Encode}(m) + \langle e, r \rangle + e_2 - \langle s, e_1 \rangle.$$

Proof. Expand $b = As + e$ and $c_1 = A^T r + e_1$; the mixed terms $\langle As, r \rangle$ and $\langle s, A^T r \rangle$ are the same sum over the same index pairs.

Lemma 4 (Compression residual). *Let $\Delta_{1,i}$ be the residual on $c_{1,i}$ and Δ_2 the residual on c_2. Then*

$$v' = \mathsf{Encode}(m) + \langle e, r \rangle + e_2 - \langle s, e_1 \rangle + \Delta_2 - \langle s, \Delta_1 \rangle.$$

Proof. Substitute $c'_{1,i} = c_{1,i} + \Delta_{1,i}$ and $c'_2 = c_2 + \Delta_2$ into $v' = c'_2 - \langle s, c'_1 \rangle$ and apply the noise identity.

Lemma 5 (Deterministic re-encryption). *If $\mathsf{Decode}(v') = m$, then Algorithm 3 recomputes $ct' = ct$ and selects ss_{ok}.*

Proof. With $m' = m$, every expansion, PKE encryption, compression, and packing step is deterministic; the fixed-length packed ciphertext equals the encapsulation value, so the equality test returns 1.

Failure probability. The working variance model is

$$\mathrm{Var}[\langle e, r \rangle] = 1024, \quad \mathrm{Var}[\langle s, e_1 \rangle] = 1024, \quad \mathrm{Var}[e_2] = 1,$$

$$\mathrm{Var}[\langle s, \Delta_1 \rangle] = 1280, \quad \mathrm{Var}[\Delta_2] = 1365.25,$$

for a total of 4694.25 and $\sigma \approx 68.5$. Rounding up to $\sigma \approx 70$ gives a working margin of approximately 14.6σ against the 1024 decision boundary. This is a heuristic, not a formal tail bound. No analytic 2^{-128} failure bound is claimed; the in-tree 32-trial test is a regression tripwire.

6.8 Intended security argument and gaps

The intended argument is FO-style: Module-LWE hides m for a passive adversary; the re-encryption check prevents ciphertext modification; the random-oracle derivation of (r, e_1, e_2) from (m, pk) prevents inconsistent randomness; implicit rejection removes a public validity oracle. The exact construction is not a verbatim copy of a standard FO transform [6,7]. In particular, the accepted secret is $\mathsf{KDF}(m \,\|\, ct \,\|\, \mathsf{H}(pk))$ and the rejection value is $\mathsf{KDF}(\mathit{fake} \,\|\, ct)$. No cited theorem covers those exact choices, and no reduction is proved here.

The lattice parameters have not been run through the community estimator. The power-of-two modulus differs from ML-KEM-1024's 3329 [1], so ML-KEM cost estimates cannot be copied. Consequently the KEM is *not* claimed IND-CCA2 secure; the target is conditional on the missing analysis.

7 Hash, XOF, and KDF layer

7.1 Functions

The hash layer exposes unkeyed BLAKE3-256, a 64-byte XOF prefix, keyed BLAKE3, the reference `derive_key` construction, an expander $\mathsf{XOF}(\mathit{seed} \,\|\, \mathit{domain})$, a KDF $\mathsf{XOF}(\mathit{domain} \,\|\, \mathit{parts}\ldots)$, and a streaming hasher. BLAKE3 is used as specified [4].

Lemma 6 (XOF prefix property). $\mathsf{H}(x)$ *equals the first 32 bytes of* $\mathsf{XOF}(x, 64)$.

Proof. This is the BLAKE3 root-bytes property, pinned by the repository tests for empty, short, chunk-edge, and multi-chunk inputs.

7.2 Framing requirements

Neither `expand` nor `kdf` inserts separators or lengths. A caller MUST ensure that its part sequence is injective. This report uses fixed-length parts for every KDF call. A new caller with variable-length parts MUST add explicit lengths or use `derive_key`.

7.3 Streaming hasher

`Hasher` supports `new`, `new_keyed`, `update`, `finalize32`, and `finalize_xof`. The first 32 bytes of `finalize_xof` equal `finalize32`. The implementation uses a fixed 54-level chaining-value stack and does not allocate.

8 Authenticated encryption

8.1 One-shot construction

Let K be a 32-byte master key and N a 12-byte nonce. Derive

$$k_e = \mathrm{DK}(\texttt{"HYP-G/v07/symm-enc"}, K),$$
$$k_m = \mathrm{DK}(\texttt{"HYP-G/v07/symm-mac"}, K).$$

Encrypt $C = \mathrm{ChaCha20}(k_e, N, 0, M)$ and authenticate

$$T = \mathrm{KeyedBLAKE3}\big(k_m,\, N \,\|\, AD \,\|\, C \\ \|\, \mathrm{LE}_{64}(|AD|) \,\|\, \mathrm{LE}_{64}(|C|)\big).$$

The one-shot opener recomputes the tag and performs a full-scan constant-time comparison before writing any plaintext. A tag mismatch or length mismatch returns `None` and leaves the plaintext buffer untouched.

The message limit is exactly 2^{38} bytes: 2^{32} ChaCha20 blocks of 64 bytes. Counter exhaustion fails closed rather than wrapping. The nonce must be unique per $(K, \text{message})$; the construction has no nonce-misuse resistance. ChaCha20 is the IETF 20-round ARX stream cipher of RFC 8439 [5].

8.2 Streaming front

The streaming sealer buffers a partial 64-byte keystream block across calls, feeds ciphertext bytes to a running keyed-BLAKE3 state, and appends the two little-endian length suffixes at `finish`. The opener feeds ciphertext to the MAC while releasing tentative plaintext; the plaintext becomes valid only when `finish` returns `Some(())`. On failure, the assembled plaintext buffer is zeroized. Empty chunks, one-byte chunks, block-boundary splits, and 1 MiB streams are tested.

Proposition 1 (Streaming identity). *For every chunking of the same (K, N, AD, M), the streaming sealer emits the same (C, T) as one-shot encryption. For every chunking of the same (K, N, AD, C), the streaming opener accepts exactly when the one-shot opener accepts and returns the same plaintext.*

Proof (Proof sketch). The subkeys, initial MAC state, and length counters are identical. The keystream state preserves the exact 64-byte block and offset across calls; by induction on the chunk sequence, the concatenated XOR bytes equal the one-shot ChaCha20 keystream with counters $0, 1, \ldots$. The running MAC absorbs exactly $N \,\|\, AD \,\|\, C$ and then the two length suffixes. The opener performs the same operations, with the additional invariant that tentative plaintext is zeroized on failure.

8.3 Failure rules

A poisoned stream never recovers. The opener's plaintext is tentative until `finish` returns $\top$; callers MUST NOT parse, display, execute, or transmit it before that point.

Condition	Sealer	Opener
length mismatch	erase chunk, poison, $\bot$	erase chunk, poison, $\bot$
counter exhausted	erase chunk, poison, $\bot$	erase chunk, poison, $\bot$
length overflow	erase chunk, poison, $\bot$	erase chunk, poison, $\bot$
finish after poison	erase tag, $\bot$	erase plaintext, $\bot$
finish length mismatch	—	erase plaintext, $\bot$
tag mismatch	—	erase plaintext, $\bot$
success	write tag, $\top$	validate, $\top$

Table 4. Streaming AEAD failure rules. $\top$ denotes `Some(())`; $\bot$ denotes `None`.

Symbol	Value
n	32 bytes (BLAKE3-256)
h	68
d	17 layers
h'	4 (16 leaves per XMSS tree)
k_{FORS}	35
a	9 (512 FORS leaves per tree)
w	16
ℓ	67 (64 message + 3 checksum digits)
pk_{sig}	64 bytes
sk_{sig}	128 bytes
sig	49,992 bytes

Table 5. Signature parameters.

8.4 Security position

The EtM structure is standard, but the exact ChaCha20 + keyed-BLAKE3 composition is not a standardized AEAD and no reduction for this exact instantiation is included. The 32-byte tag is not truncated. The construction has no key-commitment guarantee; protocols requiring key commitment must add it. Nonce discipline is entirely the caller's responsibility.

9 Hash-based signatures

9.1 Profile and sizes

The construction follows the SPHINCS+/SLH-DSA family [8,3]; XMSS is standardised in RFC 8391 [9]. HypG is not wire-compatible with either standard.

Type	Name	f_{20}	f_{24}	f_{28}
0	`T_FORS`	FORS tree	FORS leaf	zero
1	`T_WOTS_HASH`	keypair	chain	absolute step
2	`T_WOTS_PK`	keypair	L-tree height	pair index
3	`T_TREE`	FORS tree or zero	node height	node index
4	`T_ROOT`	defined, unused	defined, unused	defined, unused

Table 6. Address types and field assignments.

The sizes are:

$$\begin{aligned} \text{FORS unit} &= 32 + 9 \cdot 32 = 320, \\ \text{FORS total} &= 35 \cdot 320 = 11{,}200, \\ \text{XMSS layer} &= 8 + 67 \cdot 32 + 4 \cdot 32 = 2{,}280, \\ \text{hypertree} &= 17 \cdot 2{,}280 = 38{,}760, \\ \mathit{sig} &= 32 + 11{,}200 + 38{,}760 = 49{,}992. \end{aligned}$$

9.2 Address structure

Definition 6 (ADRS). *An address is 32 bytes:*

$$\text{layer}(4) \,\|\, \text{tree}(12) \,\|\, \text{type}(4) \,\|\, f_{20}(4) \,\|\, f_{24}(4) \,\|\, f_{28}(4),$$

all integers little-endian. The 12-byte tree field holds a 96-bit integer; valid hypertree addresses use at most 68 bits and zero the high 28 bits.

9.3 Tweakable hash functions

All three functions use keyed BLAKE3-256 with the public seed *pubseed* as the 32-byte key:

$$\begin{aligned} \mathsf{PRF}(\mathit{pubseed}, x, A) &= \text{KeyedBLAKE3}(\mathit{pubseed}, 01 \,\|\, x \,\|\, A), \\ F(\mathit{pubseed}, A, x) &= \text{KeyedBLAKE3}(\mathit{pubseed}, 02 \,\|\, A \,\|\, x), \\ \mathsf{H}(\mathit{pubseed}, A, L, R) &= \text{KeyedBLAKE3}(\mathit{pubseed}, A \,\|\, L \,\|\, R). \end{aligned}$$

The first two inputs are 65 bytes; the third is 96 bytes. The tags 01 and 02 are the DS 2.0.1 revision. H is untagged but its arity and field order are distinct.

9.4 Key generation

Algorithm 4: DS.KeyGen(*esk*)

1: *pubseed* ← CA-PRF(*esk*, `"HYP-G/v07/ds-pubseed"`, 32).
2: *prf* ← CA-PRF(*esk*, `"HYP-G/v07/ds-prf"`, 32).
3: *root* ← HT.Root(*pubseed*, *esk*), the top XMSS layer at layer 16, tree 0.
4: *pk* ← *root* ‖ *pubseed*.
5: *sk* ← *esk* ‖ *prf* ‖ *pubseed* ‖ *root*.
6: Return (*pk*, *sk*).

9.5 Deterministic randomizer and digest

Algorithm 5: PreSign(sk, msg)

1: Parse $(esk, prf, pubseed, root)$ from sk.
2: $stream \leftarrow$ CA-PRF(prf, "HYP-G/v07/ds-addrnd", 64).
3: $R \leftarrow \mathsf{H}(stream \,\|\, \mathsf{H}(msg))$.
4: Compute, for $c \in \{0, 1\}$,

$$md_c = \mathsf{H}(R \,\|\, pubseed \,\|\, root \,\|\, \mathsf{H}(msg) \,\|\, \mathrm{LE}_{32}(c) \,\|\, \texttt{"HYP-G/v07/ds-msghash"}).$$

5: Split the first 387 bits of $md = md_0 \,\|\, md_1$:

$$fors_t = \sum_{b=0}^{8} 2^b \cdot \mathrm{bit}(md, 9t + b), \quad t = 0, \ldots, 34,$$

$$tree = \sum_{b=0}^{67} 2^b \cdot \mathrm{bit}(md, 315 + b), \qquad leaf = \sum_{b=0}^{3} 2^b \cdot \mathrm{bit}(md, 383 + b).$$

6: Return $(R, fors, tree, leaf, esk, pubseed)$.

There is no fresh per-signature entropy. Signing is a deterministic function of (sk, msg), so repeated signing returns the same bytes. This is intentional for KATs and reproducibility, but it leaks message equality and removes fresh-randomness fault hedging. The scheme is *not* described as freshly hedged.

9.6 FORS

Definition 7 (FORS leaf and node). *For tree t and leaf l:*

$$A = \mathrm{ADRS}(0, 0, 0, t, l, 0), \qquad leaf(t, l) = \mathsf{PRF}(pubseed, esk, A).$$

For a node at height h and pair index p:

$$A = \mathrm{ADRS}(0, 0, 3, t, h, p), \qquad node = \mathsf{H}(pubseed, A, L, R).$$

The FORS public key is

$$\mathrm{FORS.PK} = \mathrm{KeyedBLAKE3}(pubseed, A_{\mathrm{root}} \,\|\, root_0 \,\|\, \cdots \,\|\, root_{34}),$$

where $A_{\mathrm{root}} = \mathrm{ADRS}(0, 0, 0, 0xFFFFFFFF, 0, 0)$ and $root_t$ is the root of tree t. The input is 1152 bytes. The signature for tree t is the selected leaf plus its nine authentication nodes. Verification recomputes each root from the leaf and path, then compresses the 35 roots.

Lemma 7 (FORS reconstruction). *Given a valid FORS signature for indices fors and public seed pubseed,* FORS.Verify *returns the same* FORS.PK *as FORS public-key generation.*

Proof. At height h, the verifier uses the current node index $\lfloor fors_t / 2^h \rfloor$, its sibling, and the address $(t, h, \lfloor fors_t / 2^{h+1} \rfloor)$. The signer stored exactly that sibling and the same address. Induction on h therefore reconstructs each tree root; compression is deterministic.

9.7 WOTS+ and L-tree

Definition 8 (Digits and checksum). *For a 32-byte message:*

$$d_i = (msg_{\lfloor i/2 \rfloor} \gg 4(i \bmod 2)) \wedge 0x0F, \quad i = 0, \ldots, 63.$$

The checksum is

$$c = \sum_{i=0}^{63} (15 - d_i),$$

encoded as three base-16 digits most-significant first: $d_{64} = (c \gg 8) \wedge 0x0F$, $d_{65} = (c \gg 4) \wedge 0x0F$, $d_{66} = c \wedge 0x0F$.

A WOTS+ chain applies 15 tweaked hash steps. Each step uses a type-1 address with the keypair index in f_{20}, the chain index in f_{24}, and the absolute step in f_{28}. Signing from digit d applies d steps; verification applies $15 - d$. The implementation always executes 15 calls and selects branchlessly, so the secret digit does not control the loop trip count.

Lemma 8 (WOTS chain compatibility). *For $0 \le d \le 15$, applying d steps and then $15 - d$ steps at the correct absolute addresses equals applying all 15 steps.*

Proof. Both compositions apply the same 15 functions with addresses $0, \ldots, 14$; function composition is associative and the segment boundary at d does not change the addresses.

The L-tree compresses 67 chain ends. At height h and pair index p:

$$A = \mathrm{ADRS}(layer, tree, 2, keypair, h, p), \qquad next_p = \mathsf{H}(pubseed, A, L, R).$$

If the width is odd, the final unpaired node is promoted unchanged. The widths are $67, 34, 17, 9, 5, 3, 2, 1$.

9.8 XMSS and hypertree

An XMSS layer has 16 leaves. For keypair i:

$$\begin{aligned} base &= \mathrm{ADRS}(layer, tree, 0, i, 0, 0), \\ sk &= \mathrm{WOTS.GenSK}(pubseed, esk, base), \\ pk &= \mathrm{WOTS.PKFromSK}(pubseed, sk, base), \\ leaf &= \mathrm{WOTS.LTree}(pubseed, pk, base). \end{aligned}$$

The Merkle root at height h and node index p uses

$$A = \mathrm{ADRS}(layer, tree, 3, 0, h, p), \qquad node = \mathsf{H}(pubseed, A, L, R).$$

The XMSS signature is a WOTS+ signature plus four authentication nodes. Verification recomputes the leaf and climbs with the same parity rule.

For layer $i = 0, \ldots, 16$:

$$\begin{aligned} tree_i &= tree_idx \gg 4(i+1), \\ leaf_0 &= leaf, \\ leaf_i &= (tree_idx \gg 4i) \wedge 15 \quad (i \geq 1). \end{aligned}$$

Layer 0 signs the FORS public key; each higher layer signs the root returned by the layer below. `HT.Root` is the XMSS root of layer 16, tree 0.

Lemma 9 (Hypertree consistency). *For an honestly generated key and signature, hypertree verification returns* `HT.Root`.

Proof. Induction on the layer index. The base message is the FORS public key; at layer i, XMSS reconstruction gives the next message; after layer 16 the message is the top root, which equals the public-key root by construction.

9.9 Signing and verification

Algorithm 6: DS.Sign(sk, msg)

1: $(R, fors, tree, leaf, esk, pubseed) \leftarrow$ PreSign(sk, msg).
2: Compute the 35 FORS units by signing each $fors_t$.
3: Compute the FORS public key from the units.
4: $m_0 \leftarrow$ FORS.PK.
5: For $i = 0, \ldots, 16$:
   a) $s_i \leftarrow$ XMSS.Sign$(pubseed, esk, i, tree_i, leaf_i, m_i)$.
   b) $m_{i+1} \leftarrow$ XMSS.Verify$(pubseed, i, tree_i, leaf_i, m_i, s_i)$.
6: Serialize R, the 35 FORS units, and the 17 layer signatures, inserting eight zero bytes before each layer's WOTS+ signature.
7: Return the 49,992-byte signature.

Algorithm 7: DS.Verify(pk, msg, sig)

1: If $|sig| \neq 49{,}992$, return `false`.
2: Parse R, *root*, *pubseed*.
3: Recompute the digest and split into FORS indices, tree index, and leaf.
4: Parse the 35 FORS units and compute the FORS public key.
5: For each layer, require the eight reserved bytes to be zero.
6: Climb all 17 XMSS layers in order.
7: Return ct_eq$(root', root) = 1$.

The signature encoding is

$$sig = R(32) \,\|\, \mathrm{FORS}_{35} \,\|\, \mathrm{layer}_0 \,\|\, \cdots \,\|\, \mathrm{layer}_{16}.$$

Each FORS unit is $32 + 9 \cdot 32 = 320$ bytes. Each layer is

$$\mathrm{zero}(8) \,\|\, \mathrm{WOTS}(67 \cdot 32) \,\|\, \mathrm{auth}(4 \cdot 32) = 2280$$

bytes. The total is $32 + 35 \cdot 320 + 17 \cdot 2280 = 49{,}992$. Verification parses every field and rejects nonzero reserved bytes; no field is silently ignored.

9.10 DS 2.0.1 revision

Relative to DS 2.0.0, revision 2.0.1 changes exactly two wire-visible items:

1. PRF prefixes its input with 01; F prefixes its input with 02.
2. The WOTS+ L-tree promotes an unpaired final node unchanged instead of hashing it with itself.

The first removes the 64-byte structural collision between PRF and F inputs. The second follows the FIPS 205 odd-node rule and changes every WOTS+ L-tree root, every XMSS root, and every signature. The frozen file is `kats-ds/v2.0.1.txt`; the v2.0.0 DS file remains historical.

9.11 Security position

The composition is SLH-DSA-shaped but not FIPS 205 conformant. It has no exact UF-CMA/SUF-CMA proof for its BLAKE3 instantiation, address layout, deterministic randomizer, or reserved-byte encoding. FORS is a few-time signature: security degrades as more signatures are produced under one key. The implementation enforces no signing limit and keeps no state. A deployment profile would need a concrete bound on signatures per key and a matching multi-target analysis. The current report claims only that the implementation computes the specified bytes and that the deterministic vectors and tamper tests pass.

10 Domain registry

Table 7: Domain registry. The `-init`/`-out` suffixes are CA-PRF mechanism, not separate rows.

Domain	Construction	Purpose
`HYP-G/v07/seed-a`	CA-PRF(seed, d, 32)	KEM seed_A
`HYP-G/v07/sec-s`	CA-PRF(seed, d, 512)	KEM secret s
`HYP-G/v07/sec-e`	CA-PRF(seed, d, 512)	KEM error e
`HYP-G/v07/poly-a`	CA-PRF(seed_A, d, 6144)	matrix A
`HYP-G/v07/rand-r`	CA-PRF(rseed, d, 512)	FO r
`HYP-G/v07/rand-e1`	CA-PRF(rseed, d, 512)	FO e_1
`HYP-G/v07/rand-e2`	CA-PRF(rseed, d, 128)	FO e_2
`HYP-G/v07/fake`	CA-PRF(seed, d, 32)	rejection key
`HYP-G/v07/r-seed`	BLAKE3(m ‖ pk ‖ d)	FO seed
`HYP-G/v07/ss`	XOF(d ‖ m ‖ ct ‖ H(pk),32)	accepted secret
`HYP-G/v07/reject`	XOF(d ‖ fake ‖ ct,32)	rejected secret
`HYP-G/v07/symm-enc`	BLAKE3 derive_key	AEAD enc subkey
`HYP-G/v07/symm-mac`	BLAKE3 derive_key	AEAD MAC subkey
`HYP-G/v07/ds-addrnd`	CA-PRF(prf, d, 64)	randomizer stream

Table 7: Domain registry. The `-init`/`-out` suffixes are CA-PRF mechanism, not separate rows.

Domain	Construction	Purpose
`HYP-G/v07/ds-msghash`	digest suffix	digest separation
`HYP-G/v07/ds-pubseed`	CA-PRF(esk, d, 32)	public hash key
`HYP-G/v07/ds-prf`	CA-PRF(esk, d, 32)	randomizer seed
`HYP-G/v07/kat`	hash KAT only	never protocol use

The retired draft strings `ds-fors`, `ds-wots`, and `ds-tree` are not used.

11 Implementation assurance

11.1 Memory and allocation discipline

The cryptographic libraries use fixed-size arrays and caller-provided buffers. They do not allocate, read the clock, access the filesystem, or spawn threads. The `std` crates own all OS interaction. CI greps the `no_std` libraries for `Vec`, `Box`, `alloc::`, `String`, and `std::` and fails if any appears.

11.2 Secret lifetimes

Secret-holding types redact `Debug`, avoid `Copy` where duplication would be dangerous, compare in constant time where equality is exposed, and best-effort zeroize on `Drop`. The types include the KEM secret key and shared secret, the DS secret key and pre-sign context, the AEAD key, the CA state and rule, the hash state, and polynomial buffers. This is best-effort erasure in safe Rust: normal writes are followed by compiler fences and a black-box use of the buffer pointer, but safe Rust cannot guarantee volatile erasure. The accessors `seed_copy`, `fake_copy`, and `public_copy` return caller-owned copies that the caller must erase.

11.3 Constant-time posture

The implementation avoids secret-dependent branches and indices in the cryptographic paths: KEM decapsulation always performs re-encryption and both KDFs; WOTS+ chains always execute all 15 steps; tag and root comparisons scan all bytes; implicit-rejection selection is a byte mask. This is a source-level discipline, not a machine-code guarantee. Compiler transformations and microarchitectural effects are not analysed, and no `dudect` [11] or power/EM/fault campaign has been performed.

Asset	Description
KEM shared secret	32-byte `ss` output
KEM long-term secret	`sk = seed` ‖ `fake` ‖ `pk`, especially seed/fake
KEM message	32-byte FO message, exposed as `coins`
AEAD key	32-byte master key and derived subkeys
AEAD plaintext	message and tentative streaming plaintext
AEAD associated data	authenticated but public metadata
DS master secret	`esk`, plus derived `prf`
DS public seed/root	public values; integrity matters for verification
Signature	49,992-byte encoding and every parsed field
Randomness	key seeds, encapsulation messages, nonces
Key copies	stack temporaries, buffers, caller-held copies

Table 8. Assets considered by the threat model.

11.4 Failure semantics

The KEM never returns an error; invalid ciphertexts yield a pseudorandom secret. The one-shot AEAD returns `None` and leaves the output untouched on authentication failure. The streaming opener marks plaintext as tentative until `finish`. Signature verification returns `false` on any malformation. The CLI uses exit code 2 for usage/input errors and 3 for authentication failures.

12 Threat model and failure semantics

12.1 Assets

12.2 Adversary model

The intended adversary is computationally bounded and may be classical or quantum. The model includes a network adversary that can read, modify, replay, reorder, truncate, and inject ciphertexts, signatures, nonces, and associated data; a chosen-ciphertext adversary with a decapsulation oracle; a chosen-message adversary with a signing oracle; a malformed-input adversary that supplies arbitrary fixed-length byte strings; a quantum adversary with Grover search and quantum SVP; a local software adversary that may attempt memory disclosure or compiler influence; and a fault adversary that may induce bit flips or transient faults. Fault attacks are outside the security claim, although some fail-closed paths are tested.

The adversary is not assumed to possess the long-term secrets, break BLAKE3 or ChaCha20, solve Module-LWE, run Shor's algorithm, control the operating system, or have access to a power/EM laboratory in the reported evidence.

Boundary	Trusted?	Consequence
Caller CSPRNG	Trusted	weak seeds, coins, or nonces break security
Caller key storage	Trusted	library cannot protect keys at rest
OS / allocator	Trusted	compromised OS can read or alter memory
Compiler / LLVM	Trusted	safe Rust is not a constant-time guarantee
Hardware	Trusted	cache, branch predictor, power, and EM are not modelled
Protocol framing	Caller	no boundaries, replay protection, or key confirmation
Nonce uniqueness	Caller	nonce reuse is catastrophic
Signing volume	Caller	FORS is few-time; no limit is enforced
Secret copies	Caller	copy accessors can outlive the library wipe

Table 9. Trust boundaries and caller responsibilities.

12.3 Trust boundaries

12.4 Security goals and non-goals

The KEM's intended goal is IND-CCA2 with implicit rejection; the AEAD's intended goals are IND-CPA confidentiality and INT-CTXT integrity under unique nonces; the DS intended goal is UF-CMA for a bounded signing volume. All three are *intended*, not established.

The system does not provide: a complete protocol; key storage, rotation, revocation, or HSM integration; nonce-misuse resistance; key commitment; forward secrecy by itself; replay or ordering protection; identity binding; password-based encryption; protection from a compromised RNG/OS/compiler/hardware; fault detection; or power/EM/timing resistance. It does not provide FIPS validation or NIST certification.

12.5 Failure semantics

The KEM never returns an error; invalid ciphertexts yield a pseudorandom secret. One-shot AEAD opening returns `None` and leaves plaintext untouched on authentication failure. Streaming AEAD opening releases only tentative plaintext; failure zeroizes the assembled buffer. Signature verification returns `false` on any malformation. The operator CLI uses exit code 2 for usage/input errors and 3 for authentication failures.

13 Evaluation

13.1 Method and evidence levels

We use five evidence levels: E0 source inspection; E1 deterministic tests and KATs; E2 adversarial harnesses; E3 dynamic/portability analysis such as Miri

File	Bytes	SHA-256 (prefix)
`kats/v2.0.0.txt`	28,514	`3b2dde3d...71d4b`
`kats-symm/v2.0.0.txt`	3,662	`91b7942f...4208`
`kats-hash/v2.0.0.txt`	923	`5fa77803...d9f1`
`kats-ds/v2.0.1.txt`	201,015	`e88c1bdf...c813`
Total	234,114	

Table 10. Frozen vector inventory. Complete hashes appear below.

and cross-architecture runs; E4 independent review. The current artifact has E0–E3 evidence and no E4 evidence. No result below is an external audit.

The controlled benchmark command was:

```
HYPG_BENCH_ITERS=8 \
HYPG_BENCH_DS_ITERS=8 \
HYPG_BENCH_TTEST_N=10000 \
cargo run -q --release -p hypg-bench
```

The machine was an Intel Core i5-14400F with 16 hardware threads and 31 GiB RAM, running Linux x86-64 with `rustc 1.97.1`. The benchmark warms up once and reports integer averages over eight iterations; large-message rates use three repetitions of 1 MiB.

13.2 Correctness and reproducibility

The debug, release, and MSRV 1.70 test suites each report **146 passed, 0 failed, 0 ignored**. The breakdown is 108 unit tests, 25 CLI negative integration tests, and 13 doc-tests. The red-team harness passes all 14 suites. The frozen KAT inventory is:

The complete SHA-256 values are:

```
kats/v2.0.0.txt
3b2dde3d1138c11b54a6880f50ca7b4390071ae53aa94601075f39b2af971d4b
kats-symm/v2.0.0.txt
91b7942ffb4a3c5da8b8b589268af7b7d1594796b428efdc7e26c567c5f04208
kats-hash/v2.0.0.txt
5fa778035ce2e8aa9c89345458ba4da67df4bdbe71a456cbb58026cf9341d9f1
kats-ds/v2.0.1.txt
e88c1bdf54e9390736cd03c1a165616d744a42c48d662880b344c7de3546c813
```

The x86-64 and `qemu-aarch64` static-musl KAT outputs are byte-identical across all 234,114 bytes. This checks endianness, integer width, and the DS 2.0.1 revision on an emulated ARM64 target. It is not native ARM64 validation.

Operation	Controlled result
KEM keygen / encaps / decaps	795 / 931 / 1124 μs
BLAKE3 hash / streaming / XOF (1 MiB)	842 / 837 / 827 MB/s
ChaCha20 XOR (1 MiB)	773 MB/s
AEAD seal / open (1 MiB)	427 / 434 MB/s
DS sign / verify	72.009 ms / 3.725 ms
Threaded DS sign (16 workers)	18.329 ms, 3.928×

Table 11. Controlled release measurements on the record machine. These are local point estimates, not portable guarantees.

13.3 Adversarial testing

The deterministic red-team harness covers garbage inputs, ciphertext malleability, an 8×8 key-mismatch matrix, message extremes, FO binding, sampler statistics over 524,288 CBD coefficients, failure sweeps, determinism, AEAD forgery, DS forgery, burst faults, tampered public keys, padding flips, a panic sweep, secret-key bit flips, and streaming AEAD games. A fresh release run with 20,000 trials reported 20,000/20,000 clean decapsulations. A separate full-bit run flipped all 11,520 bit positions of a test ciphertext; every flip changed the shared secret. These are regression and sanity checks, not a 2^{-128} failure proof.

13.4 Miri and portability

Miri is an interpreter for Rust's mid-level IR. Latest recorded runs on the hardened commit report KEM 9/9 (13,213.79 s), AEAD 6/6 (214.46 s), and DS bounds 3/3 (0.42 s). Full DS signing is not run under Miri because the hypertree is hours-long under interpretation. Miri checks executed paths for some classes of undefined behaviour; it does not prove their absence. The libraries also build for `aarch64`, `wasm32`, `s390x` (check only), and `thumbv7em`; KAT and red-team binaries pass under a 1 MB stack.

13.5 Performance

The valid-versus-tampered medians were 1004 per-mille for KEM decapsulation, 602 per-mille for AEAD open, and 994 per-mille for DS verify. The Welch t-test over 10,000 interleaved KEM samples had $\max |t| = 0.737$. These are timing-smoke signals, not a side-channel proof.

13.6 Interpretation

The evaluation establishes that the specified bytes are reproducible, that the tested failure paths behave as designed, that the KATs survive debug/release and cross-architecture comparison, and that the scalar implementation has usable research performance. It does not establish cryptographic security, constant-time execution, or production readiness.

Claim	Evidence	Status
KATs match the implementation	frozen files + tests	verified locally
x86-64 and aarch64 KAT bytes agree	cross-arch run	verified locally
KEM round-trip and rejection on tested inputs	unit/red-team tests	verified locally
KEM IND-CCA2	no proof; no estimator	not claimed
KEM failure $\leq 2^{-128}$	32-trial tripwire + heuristic	not established
AEAD tamper rejection on tested inputs	tests/red-team	verified locally
AEAD IND-CPA/INT-CTXT	composition argument only	not proven
DS sign/verify and tamper rejection	tests/KATs	verified locally
DS UF-CMA/SUF-CMA	no proof; no review	not claimed
Memory safety in safe Rust	forbid unsafe; Miri subsets	partially verified
Constant-time execution	branchless source patterns	not verified
Power/EM/fault resistance	none	not evaluated
NIST/FIPS compliance	none	not claimed

Table 12. Claims and evidence. E4 (independent review) is absent.

13.7 Claims and evidence matrix

14 Security analysis and open problems

14.1 Assumptions

The intended security argument depends on: (A1) Module-LWE hardness for the exact parameters; (A2) BLAKE3 as a collision-resistant hash, PRF/MAC, and XOF; (A3) ChaCha20 as a secure stream cipher with unique nonces; (A4) the exact FO-style transform having CCA security under A1–A2; (A5) random-oracle behaviour of the hash/KDF calls; (A6) a negligible decapsulation-failure probability; (A7) nonce uniqueness; (A8) a bounded DS signing volume; (A9) caller-protected secret storage; and (A10) no machine-code constant-time claim.

14.2 The exact FO variant

Standard FO analyses [6,7] do not automatically cover the chosen shared-secret and rejection derivations. A proof must adapt the transform, account for the full public-key binding in G, and bound the re-encryption-collision probability. This is an open item, not a footnote.

14.3 The DS signing volume

FORS is few-time. The security of the hypertree depends on how many messages are signed under one key and on the probability of reusing FORS leaves or

WOTS+ keys. The current implementation stores no state and enforces no limit. A deployment profile needs a concrete bound and a multi-target analysis; neither exists here. The deterministic randomizer also removes fresh-randomness fault hedging.

14.4 The AEAD composition

The EtM composition is structurally standard, but the exact ChaCha20 + keyed-BLAKE3 instantiation has no cited reduction. Nonce reuse is catastrophic and caller-controlled. No key-commitment property is claimed.

14.5 Side channels and fault injection

Source-level branchlessness is not a machine-code guarantee. The project has no `dudect` evaluation, no power/EM laboratory, and no fault-injection campaign. The deterministic DS randomizer also removes fresh-randomness fault hedging. These are material limitations for any deployment-oriented claim.

14.6 Operational assumptions

The caller must supply a CSPRNG, protect keys in memory, ensure nonce uniqueness, confirm KEM secrets, bound signature volume, and provide protocol framing and replay protection. The libraries provide none of these services.

15 Threats to validity

Construct validity. KATs and reproducibility checks measure *behaviour*, not hardness. A deterministic test can pass while the construction is insecure.

Internal validity. The red-team harness is deterministic and finite. It does not cover every input, every protocol composition, or every compiler transformation. The 20,000-trial failure sweep is not a tail bound.

External validity. Performance is measured on one CPU, one toolchain, and one operating system. Cross-architecture KATs are emulated on ARM64; native ARM64 and s390x runtime validation are absent. The results do not establish portable timing behaviour.

Conclusion validity. The absence of external audit, lattice-estimator output, formal proof, and side-channel data prevents any conclusion about deployable security.

16 Related work

ML-KEM [1], ML-DSA [2], and SLH-DSA [3] are the NIST standards closest to the relevant families. HypG shares the Module-LWE and hypertree shapes but is not wire-compatible or standards-conformant. SPHINCS+ [8] and XMSS [9] are the main references for the signature composition. BLAKE3 [4], ChaCha20 [5], and the Fujisaki–Okamoto literature [6,7] are the symmetric and transform references. The lattice-estimator [10] is the required external tool for the KEM parameters. The `dudect` [11] methodology is the minimum bar for a timing evaluation. Rust's Miri [12] is used for partial undefined-behaviour checking.

HypG's contribution is not a new hardness assumption. It is an integrated, transparent, reproducible research artifact with an explicit evidence boundary.

17 Road to a defensible profile

The shortest honest path to a deployable profile is:

1. Run the community lattice-estimator on the exact $(n, k, q, \eta) = (256, 4, 4096, 2)$ parameters, publish classical and quantum cost tables, and compare against ML-KEM-1024 without claiming equivalence.
2. Derive an analytic decapsulation-failure bound and measure it at at least 10^6 trials.
3. Prove or refute the exact FO-style transform, including the accepted and rejected secret derivations.
4. Prove a UF-CMA/SUF-CMA bound for the exact DS composition and choose an enforced signing-volume limit.
5. Complete a `dudect`-style timing evaluation and, if the threat model requires it, power/EM/fault evaluation.
6. Add coverage-guided fuzzing and a protocol-level misuse analysis.
7. Commission an independent code audit and publish findings.
8. If FIPS compliance is required, redesign around approved primitives and pursue ACVP/CAVP and CMVP validation.

Until then, HypG should remain in research, teaching, reproducibility, and test environments.

18 Conclusion

HypG v2.0 is a complete, zero-external-dependency, safe-Rust post-quantum research suite with a documented KEM, hash layer, AEAD, and stateless signature scheme. Its evaluation is unusually broad for a prototype: frozen KATs, cross-architecture equality, 146 tests, a 14-suite adversarial harness, Miri subsets, and controlled performance measurements. Its security claims are deliberately narrow: the artifact implements the specification, the tested paths behave as specified, and the remaining proof, estimator, side-channel, and audit work is named explicitly.

That combination—substantial engineering evidence and an explicit refusal to round it up into a security claim—is the contribution of this report.

A Parameter register

Table 13: Parameter register.

Object	Value	Note
KEM ring	$n = 256$, $q = 4096$	$R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$
KEM rank	$k = 4$	lattice dimension 1024
KEM noise	CBD(2)	variance 1, support $\{-2, \ldots, 2\}$
KEM sizes	$pk = 1568$, $ct = 1440$, $sk = 1632$, $ss = 32$	bytes
DS hash	$n = 32$	BLAKE3-256
DS tree	$h = 68$, $d = 17$, $h' = 4$	17 XMSS layers
DS FORS	$k = 35$, $a = 9$	35 trees, 512 leaves each
DS WOTS+	$w = 16$, $\ell = 67$	64 message + 3 checksum digits
DS sizes	$pk = 64$, $sk = 128$, $sig = 49992$	bytes
AEAD	key 32, nonce 12, tag 32	bytes
AEAD limit	2^{38}	bytes per $(key, nonce)$
CA state	32 words of 64 bits	256 bytes
CA rounds	32	fixed

B Domain and API summary

The public KEM API is `keygen(seed)`, `encaps(pk,coins)`, and `decaps(sk,ct)`. The hash API is `hash32`, `hash64`, `keyed32`, `derive32`, `expand`, `kdf`, and `Hasher`. The AEAD API is `seal`, `open`, `StreamSealer`, and `StreamOpener`. The signature API is `ds_keygen`, `ds_sign`, `ds_verify`, `pre_sign`, `serialize_sig`, and `mt_sign`.

All fallible APIs return `Option` or `bool`; they do not use exceptions. Secret-holding types redact `Debug` and best-effort zeroize on drop. The KEM `coins` argument is the FO message itself: it must be fresh for every encapsulation.

The exact signatures are:

```
keygen(seed: &[u8; 32]) -> (PublicKey, SecretKey)
encaps(pk: &PublicKey, coins: &[u8; 32])
    -> (Ciphertext, SharedSecret)
decaps(sk: &SecretKey, ct: &Ciphertext) -> SharedSecret

hash32(input: &[u8], out: &mut [u8; 32])
hash64(input: &[u8], out: &mut [u8; 64])
keyed32(key: &[u8; 32], input: &[u8], out: &mut [u8; 32])
```

```
derive32(context: &str, key_material: &[u8; 32],
         out: &mut [u8; 32])
expand(out: &mut [u8], seed: &[u8; 32], domain: &[u8])
kdf(out: &mut [u8], parts: &[&[u8]], domain: &[u8])

seal(key: &Key, nonce: &Nonce, ad: &[u8], pt: &[u8],
     ct: &mut [u8], tag: &mut [u8; 32]) -> Option<()>
open(key: &Key, nonce: &Nonce, ad: &[u8], ct: &[u8],
     tag: &[u8; 32], pt: &mut [u8]) -> Option<()>
StreamSealer::new/update/finish
StreamOpener::new/update/finish

ds_keygen(esk: &[u8; 32]) -> (SigPk, SigSk)
ds_sign(sk: &SigSk, msg: &[u8], sig: &mut [u8; 49992])
ds_verify(pk: &SigPk, msg: &[u8], sig: &[u8]) -> bool
pre_sign(sk: &SigSk, msg: &[u8]) -> PreSign
serialize_sig(r, fors_sig, ht_sig, sig_out)
mt_sign(sk: &SigSk, msg: &[u8], sig: &mut [u8; SIG_LEN])
```

C Artifact and reproducibility notes

The active documentation is in `docs/`. The frozen vectors are under `crates/hypg-kat/`. The controlled benchmark command and machine details are in `docs/metrics.md`. The test matrix and evidence levels are in `docs/testplan.md`. The exact security non-claims are in `docs/threat-model.md` and `SECURITY.md`.

The core reproducibility commands are:

```
cargo test --workspace --locked
cargo test --workspace --locked --release
cargo +1.70 test --workspace --locked
cargo run -q -p hypg-redteam
cargo run -q -p hypg-lattice
cargo test -p hypg-kat --lib --locked
HYPG_BENCH_ITERS=8 \
  HYPG_BENCH_DS_ITERS=8 \
  HYPG_BENCH_TTEST_N=10000 \
  cargo run -q --release -p hypg-bench
```

The code is licensed under GPL-3.0-or-later. The report sources are tracked in `papers/`; the PDF is built with `pdflatex`.

D Benchmark detail

The 1 MiB rates were 842 MB/s for BLAKE3 hash, 837 MB/s for streaming BLAKE3, 827 MB/s for XOF, 773 MB/s for raw ChaCha20 XOR, 427 MB/s

Operation	Measurement	Value
KEM decapsulation	valid median	1,107,761 ns
KEM decapsulation	tampered median	1,113,174 ns
KEM decapsulation	tampered/valid	1004 per-mille
AEAD open 1 KiB	valid median	3,388 ns
AEAD open 1 KiB	forged median	2,042 ns
AEAD open 1 KiB	forged/valid	602 per-mille
DS verify	valid median	3,697,178 ns
DS verify	forged median	3,678,498 ns
DS verify	forged/valid	994 per-mille

Table 14. Interleaved valid/tampered medians. These are timing-smoke signals, not a leakage proof.

Welch t-test spot	t
`c1[0]`	0.025
`c1[320]`	-0.545
`c2[0]`	0.737
`c2[80]`	0.154
`ct[last]`	0.391

Table 15. Welch t-statistics for KEM decapsulation, $n = 10{,}000$. The informational bar is $|t| > 4.5$; no spot crossed it. This is not a `dudect`-grade evaluation.

for AEAD seal, and 434 MB/s for AEAD open. The 1 KiB seal/open averages were 3/3 μs. KEM keygen/encaps/decaps averages were 795/931/1124 μs; DS sign/verify averages were 72.009/3.725 ms; the threaded signer averaged 18.329 ms with 16 workers, a 3.928× speedup.

E Comparison with FIPS 203 and FIPS 205

HypG is not ML-KEM and not SLH-DSA. The KEM uses a power-of-two modulus, different compression, a different FO-style secret derivation, and BLAKE3. The signature uses a custom 32-byte address, explicit PRF/F tags, deterministic randomizer derivation, and eight reserved bytes per layer. Matching dimensions is not a security certification.

F Symbol glossary

Micro-profile component	Measurement
BLAKE3 over 128 B	≈ 187 ns/hash
CA-PRF 1536 B	≈ 4 μs
BLAKE3 KDF 1536 B	≈ 1 μs
WOTS+ key (gen + chains + L-tree)	≈ 223 μs
XMSS layer public key	≈ 3.80 ms
FORS sign	≈ 7.77 ms
FORS verify	≈ 71 μs

Table 16. Optional single-run micro-profile (`HYPG_BENCH_PROFILE=1`, `HYPG_BENCH_ITERS=4`). These values are coarse and intended only for cost attribution.

Property	NIST reference	HypG v2.0.1
KEM ring	$q = 3329$ (ML-KEM-1024)	$q = 4096$
KEM rank	4	4
KEM pk / ct / sk	1568 / 1568 / 3168 B	1568 / 1440 / 1632 B
KEM compression	$du = 11$, $dv = 5$	$c_1 = 10$, $c_2 = 5$
KEM hash	SHA-3/SHAKE	BLAKE3
Signature shape	SLH-DSA-256f	same tree dimensions
Signature size	49,856 B	49,992 B
Signature hash	SHA-2/SHAKE	BLAKE3
Signature randomizer	hedged/randomized option	deterministic only
NIST status	standard	not standard/validated

Table 17. Dimensional similarity is not interoperability, compliance, or a security certification. The 136-byte signature difference is 17×8 reserved bytes.

Table 18: Symbols used throughout the report.

Symbol	Meaning
$\mathcal{R}_q$	$\mathbb{Z}_q[X]/(X^{256}+1)$, $q = 4096$
s, e	KEM long-term secret and error vectors
r, e_1, e_2	per-encapsulation FO randomness
m	32-byte FO message; the API's `coins`
A, b	public matrix and public vector $b = As + e$
pk, ct, sk, ss	KEM public key, ciphertext, secret key, shared secret
CBD(2)	centered binomial distribution, variance one
H	BLAKE3-256; overloaded as tweakable hash in DS
XOF	BLAKE3 extendable-output function
KDF	domain-separated XOR construction over parts
PRF, F, H	DS tweakable hash functions
R	32-byte DS randomizer
md	64-byte DS message digest
ADRS	32-byte DS address

Table 18: Symbols used throughout the report.

Symbol	Meaning
FORS	few-time forest of 35 height-9 trees
WOTS+	Winternitz one-time signature, $w = 16$, $\ell = 67$
XMSS	Merkle tree over 16 WOTS+ leaves
h, d, h'	total height, layers, per-layer height
sig	49,992-byte signature
σ	standard deviation; in KEM context, working noise scale

References

1. National Institute of Standards and Technology. *Module-Lattice-Based Key-Encapsulation Mechanism Standard*. FIPS 203, August 2024.
2. National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. FIPS 204, August 2024.
3. National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard*. FIPS 205, August 2024.
4. J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn. BLAKE3: one function, fast everywhere. Technical report, 2020. `https://github.com/BLAKE3-team/BLAKE3-specs/`.
5. Y. Nir and A. Langley. ChaCha20 and Poly1305 for IETF Protocols. RFC 8439, June 2018.
6. E. Fujisaki and T. Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In *CRYPTO 1999*, LNCS 1666, pages 537–554, 1999.
7. D. Hofheinz, K. Hövelmanns, and E. Kiltz. A modular analysis of the Fujisaki–Okamoto transformation. In *TCC 2017*, LNCS 10677, pages 341–371, 2017.
8. D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe. The SPHINCS+ signature framework. In *ACM CCS 2019*, pages 2129–2146, 2019.
9. A. Hülsing, D. Butin, S.-L. Gazdag, J. Rijneveld, and A. Mohaisen. XMSS: eXtended Merkle Signature Scheme. RFC 8391, May 2018.
10. M. R. Albrecht, R. Player, and S. Scott. On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology*, 9(3):169–203, 2015.
11. O. Reparaz, J. Balasch, and I. Verbauwhede. Dude, is my code constant time? In *DATE 2017*, pages 1697–1702, 2017.
12. The Rust Project. Miri: an interpreter for Rust's mid-level intermediate representation. `https://github.com/rust-lang/miri`.